



Toward Design and Architecture-Level Intelligence

Mirosław Staron^{ID} and Silvia Abrahão^{ID}

WHEN CODE BECOMES cheap, design and architecture become more valuable. When generative AI gives us the ability to get almost anything implemented in no time, the job of software engineers becomes extremely interesting. We can increasingly focus on what we have been trained to do—software design, software architecture, and software construction. The skills and craftsmanship of software engineers now form the critical boundary between “vibe-coding” and professionally engineered software.

In this column, we cover papers from the 2025 International Conference on Software Architecture (ICSA), focusing on high-quality software design, emerging architectural paradigms and their analysis methods, and software architecting more broadly. We welcome your feedback and suggestions on the topics covered. Additionally, if you experiment with or adopt any of the practices featured here, please share your experiences with us, and the authors of the respective papers.

Software architecting involves understanding the constraints under which a system must operate and

creating a set of components that, together with their interactions, fulfill these constraints. Although this may sound a straightforward task, it requires years of experience and a specialized skill set. Software architects must be able to identify the appropriate nonfunctional requirements (as functional requirements are typically provided by business analysts), decide on the implementation technologies (e.g., Java or C#), and identify the most important tradeoffs among quality attributes (e.g., performance versus portability). They must also ensure that design principles are followed throughout implementation. **Figure 1** presents the distribution of research topics at the 22nd IEEE International Conference on Software Architecture (ICSA 2025), held in Odense, Denmark. Although the main conference theme emphasized intelligent system design, many contributions focused on the craftsmanship of software architects.

Most papers addressed some form of architectural analysis, either by proposing new methods or applying existing ones to emerging architectures. The second-largest group focused on new design and architectural patterns, including their evaluation in the context of intelligent systems (the conference’s main

topic). Finally, several papers explored sustainability, architectural properties, and novel component types for intelligent systems. In this column, we examine studies that advance analysis and decision-making at the design and architectural levels.

First Steps Toward Design-Level Intelligence

A recurring question in industry discussions about large language models concerns the quality of the designs they produce. Industrial software engineering is interested in code generation, but to make a sustainable business of it, we need good design. So, the question of whether the models “understand” design is valid. In their paper “Do Large Language Models Contain Software Architectural Knowledge? - An Exploratory Case Study with GPT,” Mohamed Soliman and Jan Keim assess GPT-3.5’s ability to answer questions about architectural knowledge. They posed architecture-specific questions about the Hadoop distributed file system (HDFS) to GPT and compared the AI’s responses against a verified ground truth of architectural documentation. To make the study relevant, they test the model’s ability to reason about quality attributes and design rationales of a

complex, real-world system (HDFS). The results showed moderate recall (0.57–0.62) but quite low precision (0.33–0.51). While GPT 3.5 could identify basic architectural components, it struggled with the “why” (design rationales) and the specific “how” (quality attributes in particular). The results include several deep dives, e.g., a finding that GPT can correctly identify more than half of a subsystem’s components but is less precise in identifying connectors. The practical takeaway is that GPT 3.5 can be useful for “initial discovery” or summarizing known systems; it should probably not yet be trusted for critical design decisions without expert human validation. The paper can be found here: <https://tinyurl.com/y8mdzr2b>.

GenAI for Design-Level Intelligence, but Where Exactly?

A quote from William Gibson (the author of *Neuromancer*) famously said: “The future is already here—it’s just not evenly distributed.” This is very true for the field of design and architecture, where the generative AI technology can help in some areas better than in others. In their paper “Will Generative AI Fill the Automation Gap in Software Architecting?” James Ivers and Ipek Ozkaya address this question by studying where generative AI can be useful in software architecture. The paper introduces a framework for evaluating GenAI fit by comparing its core competencies (brainstorming, summarization) with its limitations in software engineering (objective analysis, fact-checking, and contextualization). The authors conducted a reflective assessment of five core architecture activities: defining architecturally significant requirements (ASRs), designing the

architecture, evaluating the architecture, documenting the architecture, and reconstructing the architecture. They found that GenAI is well-suited for brainstorming-heavy tasks, such as identifying stakeholders, generating well-formed ASRs, and identifying design alternatives. On the contrary to that, GenAI is poorly suited for tasks requiring “high-fidelity representation” or “objective analysis,” such as assessing the correctness of views, prioritizing requirements, or identifying conflicts between decisions. These results allow us to reshape the role of the software architect to engineer software even more—architects should be responsible for analysis and take advantage of GenAI when testing their ideas or supporting tasks like stakeholder identification. The paper can be accessed at: <https://tinyurl.com/bdp53uhs>.

Patterns and Antipatterns: Somewhere There is a Balance

The static properties of software and its dynamic properties are not always aligned. A monolith can be faster than a structure of microservices,

but the microservices are definitely more maintainable than a monolith. In their paper “Architecture and Performance Anti-patterns Correlation in Microservice Architectures,” Alberto Avritzer, Andrea Janes, Catia Trubiani, Helena Rodrigues, Yuanfang Cai, Daniel Sadoc Menasche, and Alvaro Jose Abreu de Oliveira test the correlation between structural architecture antipatterns (static) and performance antipatterns (dynamic). The study uses a unified model that integrates heterogeneous dependencies—specifically, service calls and entity sharing—to visualize architectural issues using a design structure matrix (DSM). The study uses two state-of-the-art tools: DV8 for detecting architectural antipatterns (such as “Cliques” and “Unstable APIs”) and PPTAM for identifying performance bottlenecks (such as “Blobs” and “Traffic Jams”). The results demonstrate that performance testing based on operational profiles can help architects prioritize which parts of the system to refactor first to achieve the greatest performance gains. The practical implication is, among others, to

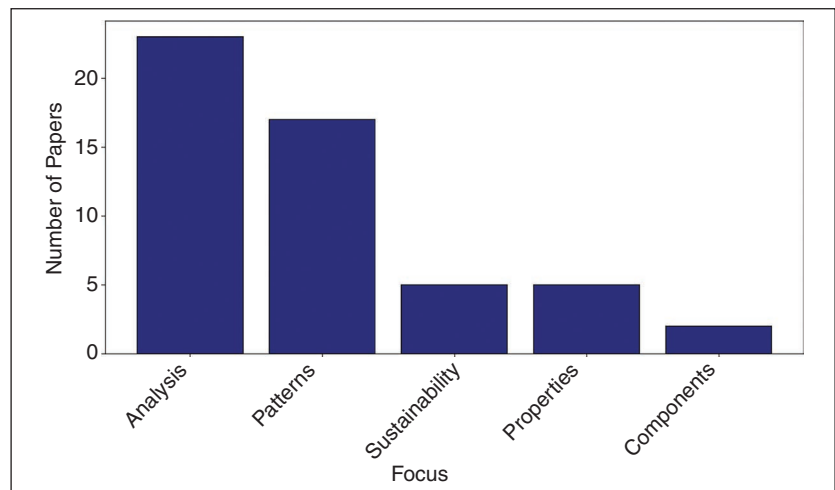


FIGURE 1. Distribution of papers' focus at ICSA 2025.

combine static architecture analysis with dynamic performance monitoring to identify root causes of bottlenecks that neither tool could find in isolation. The paper can be accessed at: <https://tinyurl.com/5n8zvcvt>.

analysis (Spearman's ρ) to examine relationships between centrality and traditional software metrics. The results show that centrality metrics are only weakly to moderately correlated with conventional metrics,

The skills and craftsmanship of software engineers now form the critical boundary between “vibe-coding” and professionally engineered software.

Not All Services Are Equal: A Centrality View of Microservices

Understanding the internal structure and quality of microservice architectures remains a persistent challenge, particularly when traditional software metrics fail to capture system-level dependencies. In their paper “Network Centrality as a New Perspective on Microservice Architecture,” Alexander Bakhtin, Matteo Esposito, Valentina Lenarduzzi and Davide Taibi explore whether graph-based centrality metrics can provide complementary insights into microservice design and help identify architectural issues. The authors conduct an empirical study on 24 open source Java Spring microservice systems, analyzing 53 services. They reconstruct service dependency graphs using static analysis (Code2DFD) and compute a comprehensive set of metrics, including size, complexity, quality, and multiple network centrality measures (e.g., degree, betweenness, eigenvector). They then apply statistical correlation

suggesting that they capture different architectural properties. Notably, central services tend to expose more public application programming interfaces, exhibit lower internal complexity, and contain fewer code quality issues. However, these relationships are not strong enough to replace traditional metrics. Instead, centrality offers an orthogonal perspective, particularly useful for identifying architectural antipatterns such as hub-like or overly dependent services. For practitioners, this work highlights that incorporating network-based analysis can enrich architectural assessments, offering a more system-level understanding beyond code-centric metrics. The paper can be accessed at: <https://tinyurl.com/2zmky6py>.

Granularity Matters: Energy and Performance in Microservices

Microservice granularity—how functionality is distributed across services—is a key architectural decision, yet its impact on energy

consumption remains largely unexplored. As sustainability becomes a growing concern in software engineering, understanding these tradeoffs is increasingly important. In their paper “How Does Microservice Granularity Impact Energy Consumption and Performance? A Controlled Experiment,” Zhao et al. investigate how different levels of granularity affect both energy consumption and performance through a controlled experiment. The authors analyze two open source systems of different sizes, creating coarse, medium, and fine-grained versions of each. Using a goal-question-metric (GQM) approach, they subject these systems to varying request loads (40–400 requests/second) and measure energy consumption and response time using a dedicated experimental infrastructure. The results show that finer granularity generally increases energy consumption, as distributing functionality across more services introduces overhead. However, poorly designed coarse decompositions can also lead to higher energy use due to overloaded services. Performance differences are more modest: Coarser granularity tends to yield slightly better response times, while request load has a much stronger impact than granularity itself. Interestingly, no strong tradeoff between energy and performance was observed—both can improve together under appropriate designs. One important caveat to this is that such a tradeoff could indeed manifest with horizontal scaling to accommodate much higher request loads, which was out of scope for this study. For practitioners, this study highlights that granularity decisions should be guided not only by maintainability or scalability, but also by energy

considerations, especially in large-scale systems and high-load scenarios. The paper can be accessed at: <https://tinyurl.com/5n77v2wa>.

Quantifying Sustainability in Software Architecture Decisions

Sustainability is becoming an important concern in software systems, yet evaluating its impact at the architectural level remains challenging due to complex interdependencies among quality attributes. In their paper “Using Sustainability Impact Scores for Software Architecture Evaluation,” Fatima et al. introduce an enhanced sustainability impact score (SIS) to support architecture evaluation by quantifying how design decisions affect multiple sustainability dimensions. Building on prior work, the

authors propose a refined SIS model that incorporates risk- and importance-based prioritization of quality attributes, and tradeoff effects supported by artifacts such as decision maps and dependency matrices. The approach is evaluated through a case study in the energy domain, comparing two alternative architectural solutions (i.e., alternative deployment and functional configurations) and their tradeoffs across sustainability dimensions. The results show that architectural decisions produce not only direct effects but also hidden enabling and systemic impacts across dimensions. The SIS helps reveal these tradeoffs, showing, for example, that on architecture configuration better supports economic and social sustainability, while both exhibit challenges in environmental

impact. The use of normalization and theoretical optimal baselines further enables meaningful cross-dimension comparisons. For practitioners, this work provides a structured and relatively lightweight approach to make sustainability tradeoffs explicit, supporting more informed and forward-looking architectural decisions. The paper can be accessed at: <https://tinyurl.com/mr2m8fzw>.

Our digest highlights how software architecting is augmented by intelligent tools that can reason at the level of components, patterns, and nonfunctional aspects such as sustainability. Across the six selected studies from ICSA 2025, a consistent picture emerges: While generative architecture-level intelligence introduces new opportunities—particularly for exploration and

Table 1. Insights from selected ICSA 2025 papers.

Research Focus	Key Insights	Takeaways
LLMs and architectural knowledge	GPT-3.5 shows moderate recall but low precision when reasoning about architecture; it identifies components but struggles with design rationale and quality attributes.	Use LLMs for exploration and documentation support but rely on expert validation for architectural decisions.
GenAI support in architecting	GenAI is effective for brainstorming and generating alternatives, but weak in tasks requiring precision, validation, and contextual reasoning.	Treat GenAI as a copilot for ideation; retain human responsibility for analysis, prioritization, and decision-making.
Correlation of architectural and performance antipatterns	Combining static (architecture) and dynamic (performance) analyses can help reveal bottlenecks that neither approach detects alone.	Integrate structural analysis with runtime monitoring to better prioritize refactoring efforts.
Network centrality in microservices	Centrality metrics capture system-level dependencies that traditional metrics do not reflect; they highlight critical or highly central services.	Complement traditional metrics with graph-based analysis to detect architectural risks such as hub services.
Microservice granularity versus energy/performance	Finer granularity increases energy consumption due to overhead; performance differences are smaller and workload-dependent.	Consider energy efficiency when deciding service granularity, especially in high-load or large-scale systems.
Sustainability-aware architecture evaluation	SIS reveals direct and indirect effects of architectural decisions across multiple dimensions.	Use structured methods like SIS to explicitly assess sustainability tradeoffs and support informed architectural decisions.

ABOUT THE AUTHORS



SILVIA ABRAHÃO is a full professor in the Department of Computer Science, Universitat Politècnica de València, València 46022, Spain. Contact her at <https://sabrahao.wixsite.com/dsic-upv> or sabrahao@dsic.upv.es.



MIROSLAW STARON is a full professor in the Interaction Design and Software Engineering division, Department of Computer Science and Engineering, Chalmers University of Technology and the University of Gothenburg, Gothenburg 412 96, Sweden. Contact him at <https://www.staron.nu> or mirosław.staron@cse.gu.se.

ideation—its limitations reinforce the central role of the architect's human craftsmanship. At the same time, advances in analytical techniques, such as combining static and dynamic views, leveraging network-based metrics, and incorporating sustainability assessment models, provide architects with richer and more holistic insights into complex systems. As summarized in [Table 1](#), the studies offer complementary perspectives on how to better analyze, evaluate, and reason about architectural decisions in practice, ultimately helping practitioners make more robust, sustainable, and context-aware design choices. 

computer.org/pervasive



Call for Articles

IEEE Pervasive Computing publishes accessible, useful peer-reviewed papers on the latest developments in pervasive, mobile, and ubiquitous computing. Topics include hardware technology, software infrastructure, real-world sensing and interaction, human-computer interaction, and systems considerations, including deployment, scalability, security, and privacy.



Author Guidelines
bit.ly/3UoGDT7